

MADE BY:

- Antoine Berthion (566199)
- Capucine Speilers (540555)
- Andrius Ežerskis (542698)
- Nha Truong (576343)

INFO-H415

Advanced databases

PROJECT:

Graph databases
with Neo4j and ArangoDB

2025 - 2026

INDEX

of content

01

Introduction

02

Technology choice

03

Tools choice

04

Dataset and benchmarks

05

Testing and results

06

Conclusion

INTRODUCTION

Goal of the project

- Benchmark Neo4j and ArangoDB
- Justify the use of this specific technology in the context of our application

What is a benchmark?

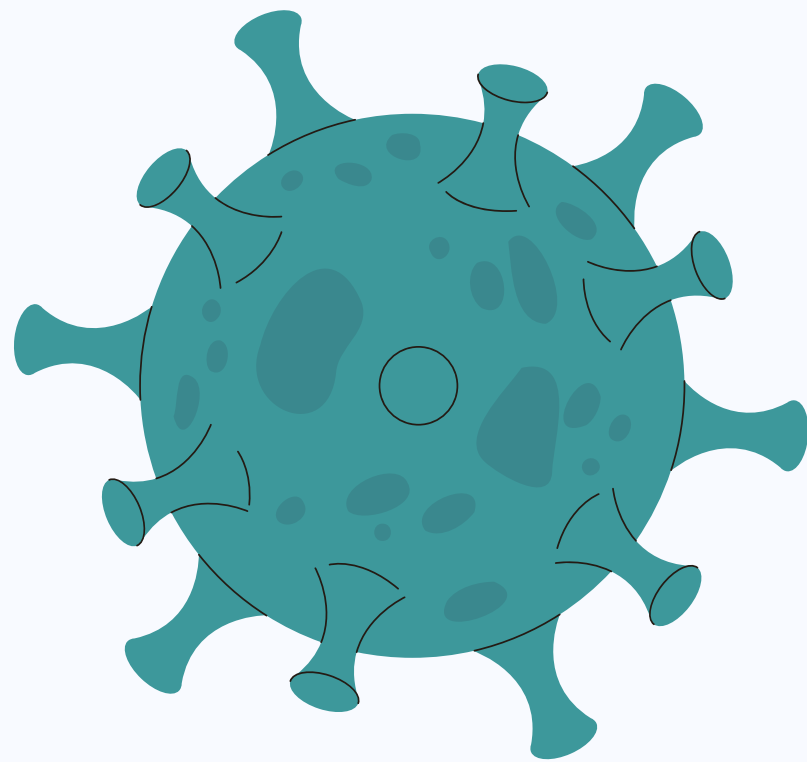
- Standardized comparison of tools
- Ensure reproducibility

Why a graph database?

Specific use case requires specific tools

- Social network
- Recommendation systems
- Fraud detection

OUR APPLICATION



2. TECHNOLOGY CHOICE

2.1 TECHNOLOGY CHOICE

Graph database

GRAPH DATABASE

Description

Graph databases use graph-based structures with attributes, relationships and objects to represent data, allowing for a better representation of connected data.

As the data is represented by a graph, traversing the data through their relationships becomes trivial and efficient, with no need to join tables or create temporary data structures.

GRAPH DATABASE

Theoretical notions

Oriented graph

An oriented graph is defined as a pair:

$$G = (V, E)$$

where V is a finite set of vertices, and $E \subseteq V \times V$ is a set of oriented edges

Nodes and relationships

- Nodes (vertices) represent entities (discrete objects) of a domain
- Relationships (edges) describe how nodes are connected

Attribute functions

$$\phi : V \rightarrow \mathcal{A}_V$$

$$\phi : E \rightarrow \mathcal{A}_E$$

where $\mathcal{A}_V$ and $\mathcal{A}_E$ denote respectively the sets of possible attributes for nodes and relationships

GRAPH DATABASE

Theoretical notions

Properties

- Key-value pairs used to store data on both nodes and relationships
- Link to the notion of multiple-entry tables in classical databases

Traversals

A query corresponds to traversing a graph by visiting nodes V and following relationships E according to specific rules, only a subset of the graph is visited

Neighborhood

The minimal step in traversals if going from one node to its immediate neighbors

GRAPH DATABASE

Theoretical notions

Paths

More complex traversals consist of following a sequence of edges

k-hop neighbors

A vertex u is a k-hop neighbor of v if there exists a path of length k from v to u

k-hop neighborhood

The set of all k-hop neighbors of v is called its k-hop neighborhood

GRAPH DATABASE

Strengths

High connectivity data

Graph databases allow to handle highly connected data very effectively, as the structure of graphs makes it easy to represent relationships.

Graph traversal

Tasks such as fraud detection, social network analysis and recommendation systems benefit from the ability to traverse graphs efficiently.

Adaptability

The flexible nature of graphs leads to being able to easily adapt and change the structure or schema of the graph database, which is crucial in a constantly evolving environment.

GRAPH DATABASE

Limitations

Large values

Graphs databases are not meant to handle storing entity properties with extremely large values, such as BLOBs (Binary Large Objects) or CLOBs (collections of character data).

Scaling

Designed for single-server architectures, graph databases may handle horizontal scaling poorly. Vertical scaling is possible, but writes to the database are still limited.

2.2 TECHNOLOGY CHOICE

Relational database

RELATIONAL DATABASE

Description

Relational databases use tables where columns correspond to specific attributes of the data being stored, and rows correspond to individual pieces of that data.

In order to retrieve data, relational databases often need to take into account multiple tables, by performing joins and creating a temporary structure with the result we want.

RELATIONAL DATABASE

Strengths

Centralized storage

Data is stored in tables with a fixed schema. This structure allows for a consistent format and makes it easy to store and retrieve data

Consistency

Primary and foreign keys, as well as unique constraints, allow to keep the data consistent, preventing the entry of invalid or redundant data

SQL

SQL is the language that allows relation databases to be queried and managed. Operations such as insertions, deletions and updates can be performed

RELATIONAL DATABASE

Limitations

Data size

As the data grows larger, it becomes difficult, for relational databases, to keep up with the more complex queries that are needed to extract useful information.

Flexibility

As relational databases work with a fixed schema defined at database creation, it can require significant changes to applications and/or downtime.

2.3 TECHNOLOGY CHOICE

Graph vs relation databases

GRAPH VS RELATION DATABASES

Example

Situation

Suppose we have:

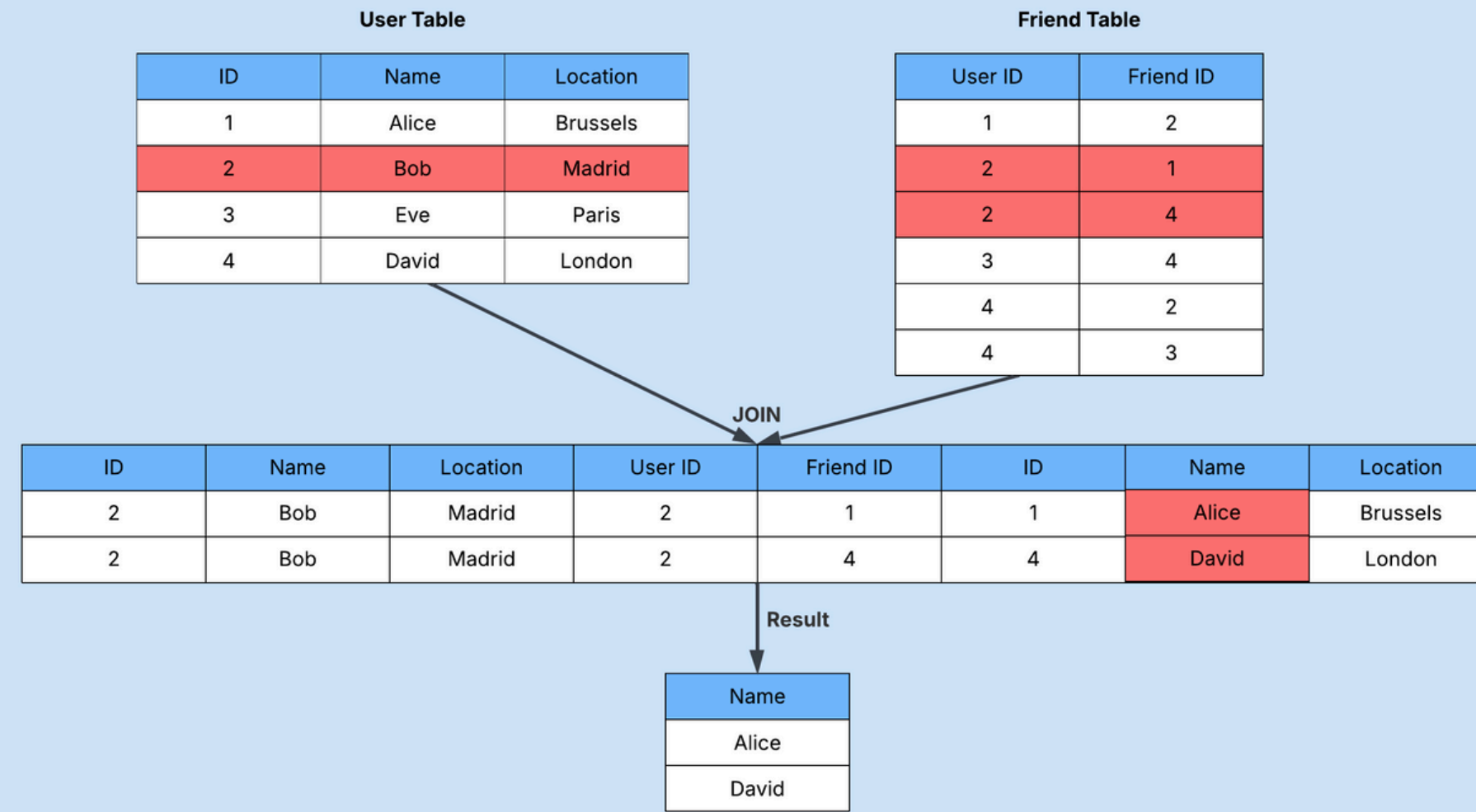
- a user table containing the ID, name and location of the users of some service
- a friends table containing pairs of user IDs representing friendships

What are the names of Bob's friends?

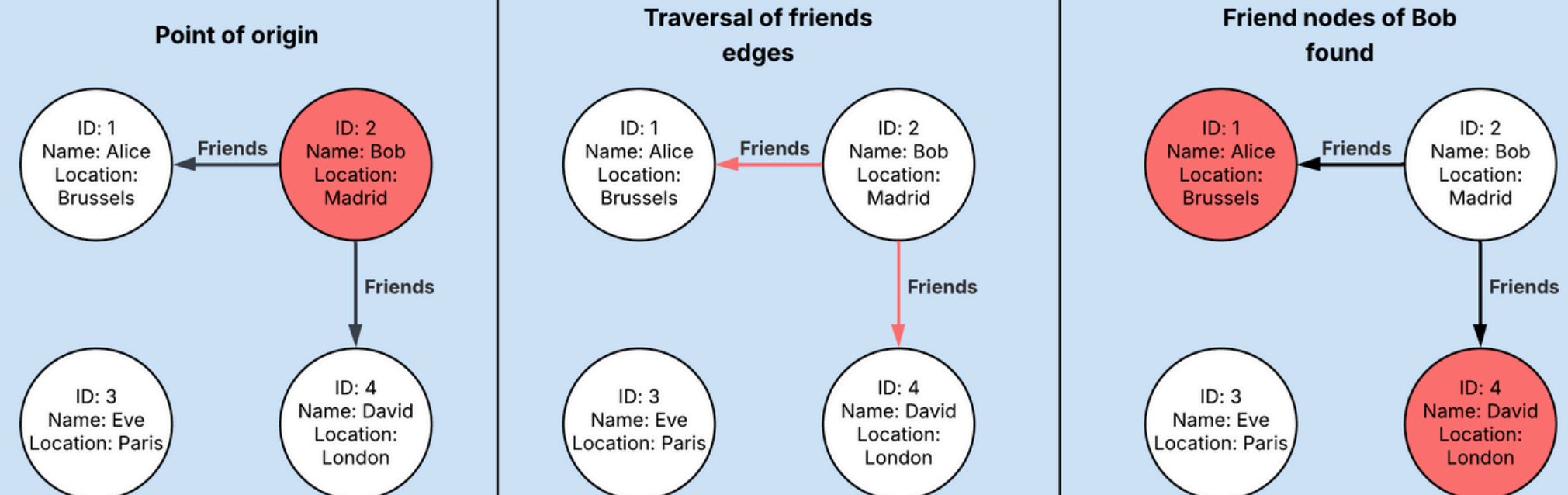
1. Find Bob's row in the users table to retrieve his ID
2. Find every row in the friends table where Bob's ID appears
3. Join each friend ID from those rows back to the users table

What are the names of Bob's friends ?

RELATIONAL DATABASE



GRAPH DATABASE



GRAPH VS RELATIONAL DATABASES

Observation

Solving that simple query

Relational Database

- Requires multiple table scans and joins
- Leads to a large and expensive data structure

Graph Database

Equivalent to finding the 1-hop neighborhood of Bob (the number of joins grows linearly with k the number of hops).

As the depth of our queries increases, the memory consumed for intermediate results will grow rapidly and impact performance very easily.

3. TOOLS CHOICE

3.1 TOOLS CHOICE

Neo4j

NE04J

Description

Native graph database
management system (graph
model to the storage level)

Queries are expressed using Cypher, a declarative query language similar to SQL, but specially designed and optimized for graphs

- Deliver fast responses even for complex queries
- Scale efficiently to support growth in data and users
- ACID compliance

NEO4J

Strengths

DBMS popularity indices

- Currently the most widely recognized and adopted graph database system
- Richer ecosystem of learning resources and community support

Native graph database

- Single-model design
- Storage engine, data model, and query mechanisms are built specifically for graph structures

Graph model

- Store data as key-value pairs on both nodes and relationships
- Neo4j Browser: environment to visualize and interact with graph data

NEO4J

Limitations

Sharding support

Some queries and operations perform very poorly on sharded Neo4j databases, and only work with some specific versions of the Cypher query language.

Scale reads

- Only the master instance in the cluster can handle the writes
- Slaves have to forward write requests to the master node

3.2 TOOLS CHOICE

GrangoDB

ARANGODB

Description

Multi-model database system
that supports different data
models within a single engine

- JSON documents
- Key-value pairs
- Graphs (node and edge collections)

Queries are expressed in AQL, mainly a declarative language designed to work uniformly across all supported data models and allows the combination within a single query

AQL provides native constructs for graph navigation, allowing multi-hop traversals to be expressed directly in the query language:

```
FOR v, e, p IN 1..k OUTBOUND s GRAPH G RETURN v.
```

ARANGODB

Strengths

Multi-model system

- It can handle multiple data models, such as documents, key-value pairs, and graphs
- Strong adaptability as the data requirements for a project can change and evolve

AQL language

- Allows for unified queries to handle the different types of data
- Excels at managing interconnected data, with multiple complex relationship mappings

Resource utilization

Managing all data models with a single engine optimizes resource utilization, leads to reduces costs for infrastructure and simplifies database management.

ARANGODB

Limitations

Smaller community

As less people are interesting in using ArangoDB, it means there are less features added overtime, as well as less documentation is available, which can be problematic when discovering and using the tool.

Limited integrations

Integrations, like GraphQL, are limited, and the schema of the database needs to be generated manually, instead of using the existing schema of the graph database.

4. DATASET AND BENCHMARKS

4.1 DATASET AND BENCHMARKS

Dataset

DATASET

Choice

nodes/edges

- 10k / 30k
- 30k / 150k
- 1M / 3M

Online datasets

- Too large / heavy cleaning needed
- Reducing the size while keeping the structure meaningful is hard

Generated structure

- Social graph during Covid
- Nodes = people (name, age, has_covid, country)
- Edges = relation degree (exposure)

Design choices

- Keep a relatively consistent ratio nodes/edges
- From small to larger scale
- Structure allowing graph traversal and filtering

4.2 DATASET AND BENCHMARKS

Queries

QUERIES

Writing

Data insertion

- Fundamental for databases
- Reflects how efficiently the system handles data ingestion and, in our case, graph construction

Measuring insertion time

- Allows us to assess write performance and the impact of graph size on bulk data loading
- Relevant for dynamic use cases with frequent graph updates

QUERIES

Graph traversals

QUERY 1: COUNT PEOPLE WHO HAVE AT LEAST ONE CONTACT WITH COVID

QUERY 2: COUNT PEOPLE WITH NO COVID CASES IN THEIR 2-DEGREE CIRCLE

Queries 1 and 2 covers 1-hop
and 2-hops neighborhood
sensing

Evaluate how efficiently each
database handles multi-level
relationship exploration

QUERIES

Filtering

QUERY 3: COUNT HEALTHY PEOPLE AT RISK FROM CLOSE CONTACTS

QUERY 4: COUNT COVIDED PEOPLE OF AGE 65+

Edge filtering

By restricting query 3 to relationships with particular properties, see how efficiently each database handles relationship filtering

Node filtering

Query 4 allows us to evaluate a simple node filtering query

QUERIES

Updating / Removing

QUERY 5: CONTAMINATE PEOPLE IN RELATIONSHIPS WITH COVID CASES

QUERY 6: KILL PEOPLE WITH COVID

Graph updating

Query 5 updates nodes properties, allowing us to evaluate the writing capabilities of each tool

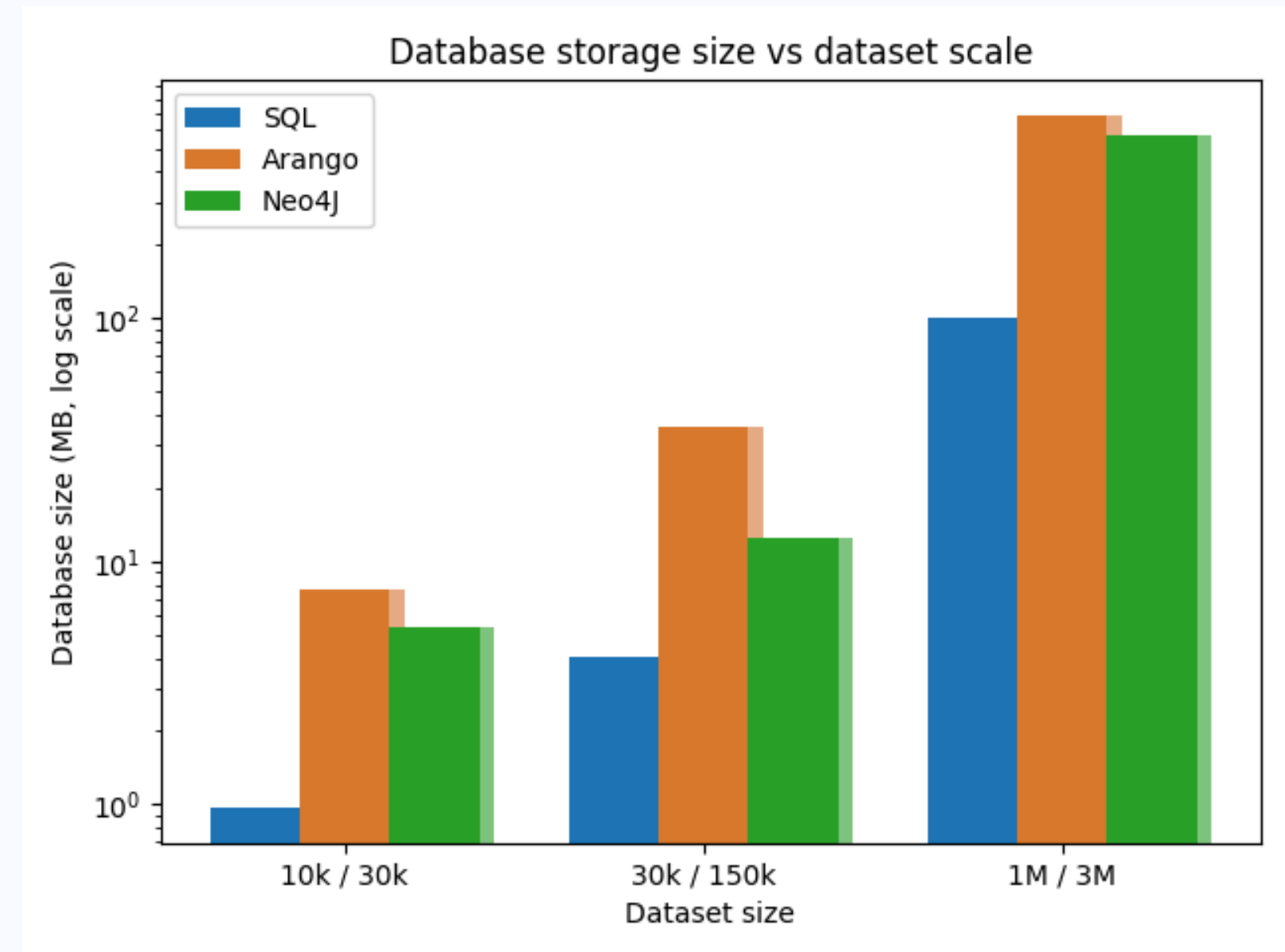
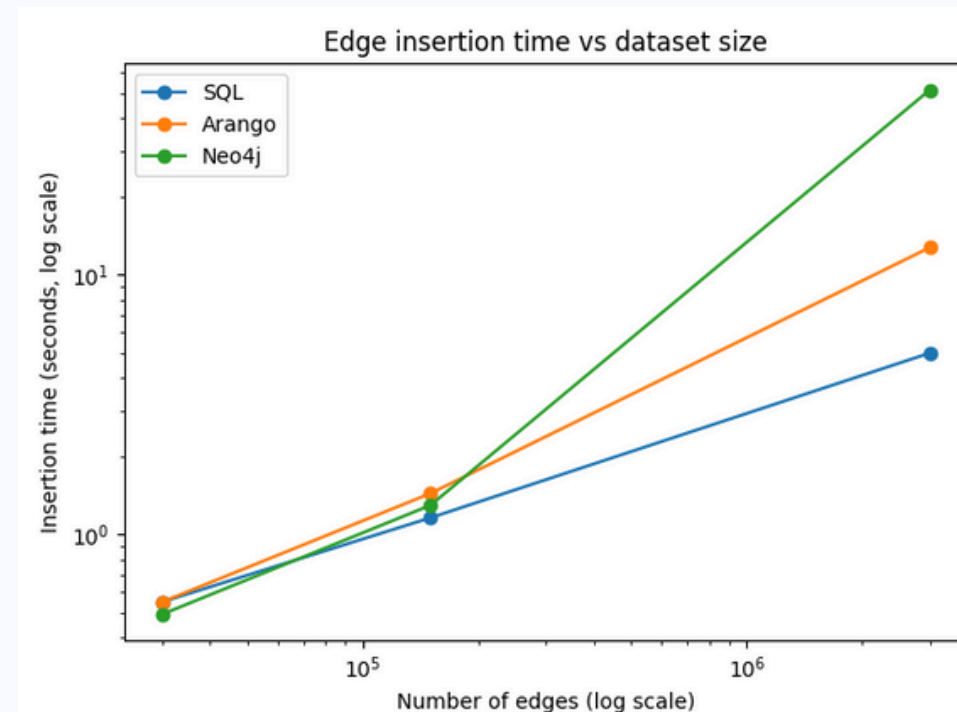
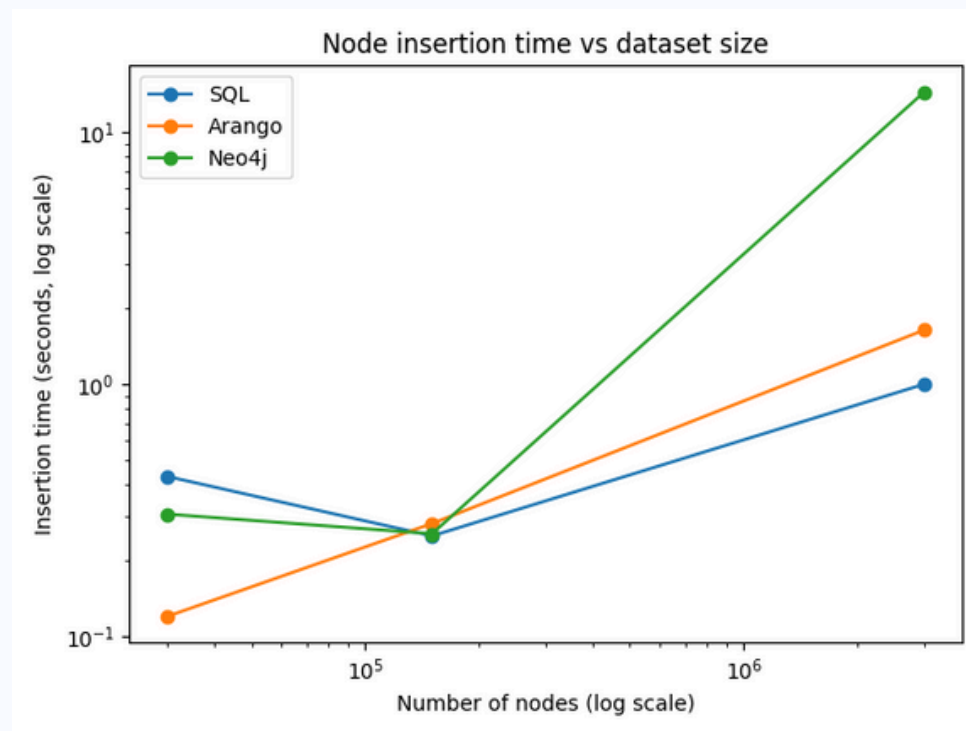
Node removal

Query 6 tests the efficiency of tools with item removal

5. TESTING AND RESULTS

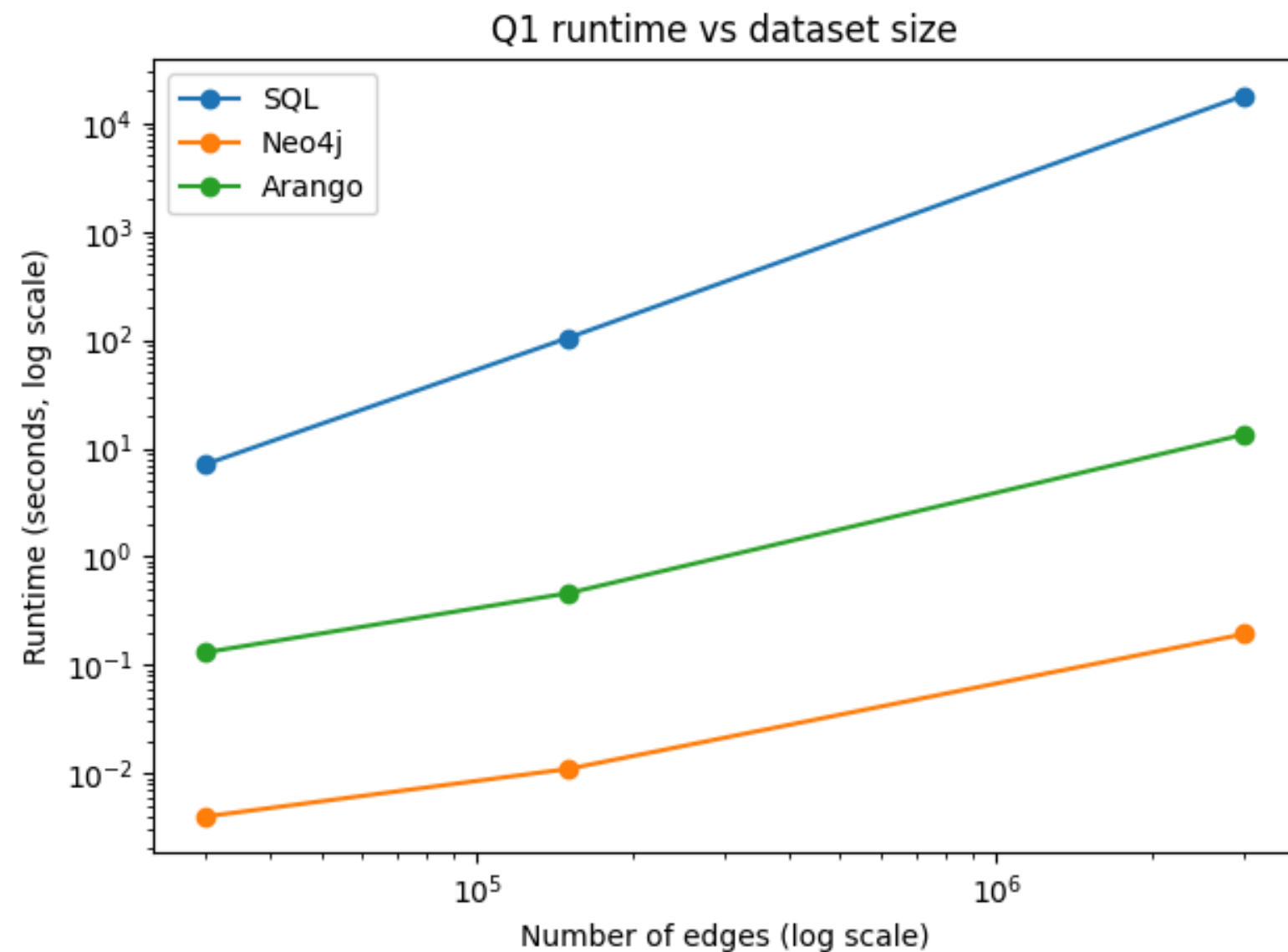
INSERTION

INSERTION TIME AND SIZE



QUERY I

COUNT PEOPLE WHO HAVE AT LEAST ONE CONTACT WITH COVID

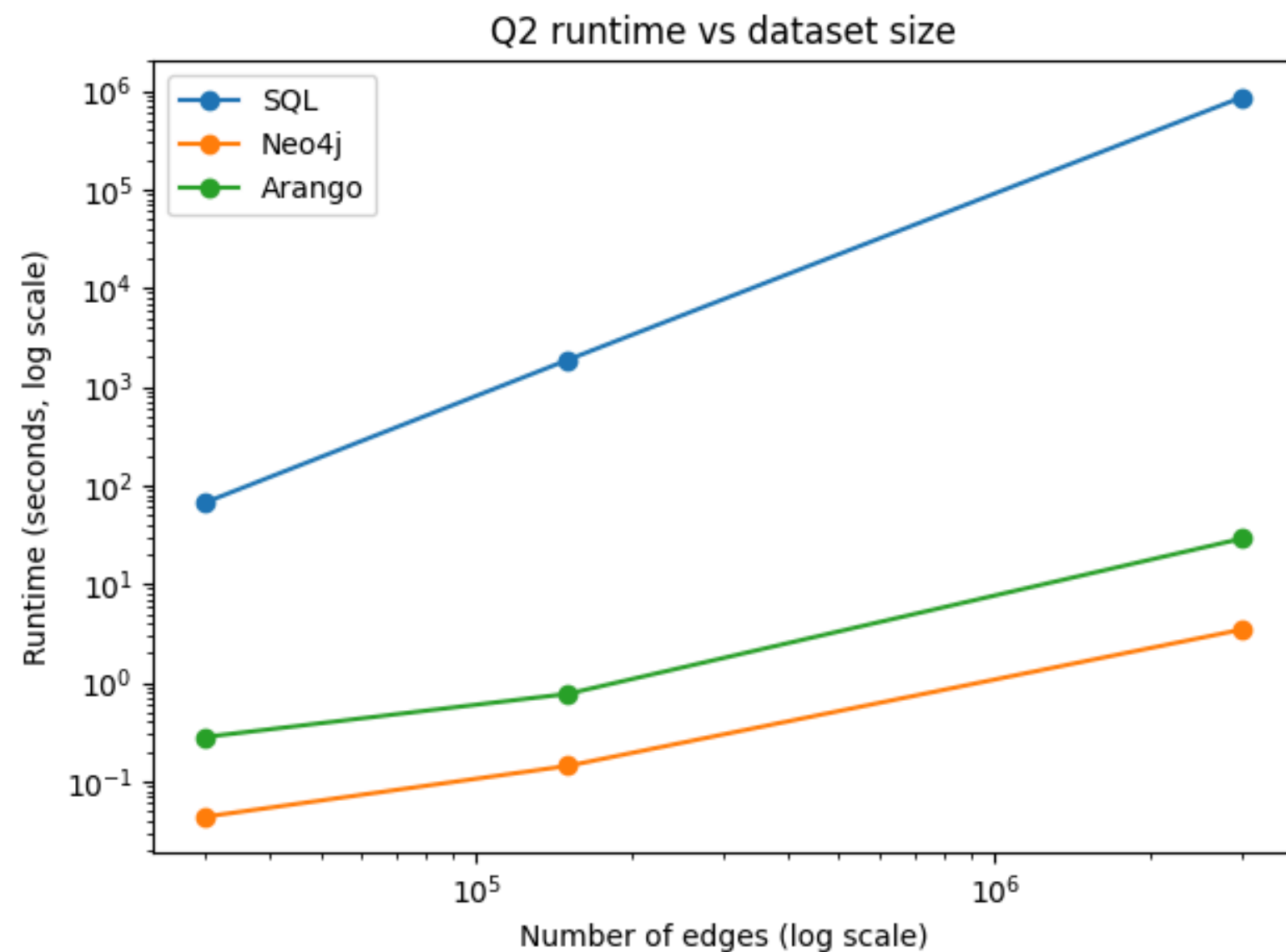


Discussion

- SQL is nearly 100 to 1000 times slower than ArangoDB
- ArangoDB is about 100 times slower than Neo4j

QUERY 2

COUNT PEOPLE WITH NO COVID CASES IN THEIR 2-DEGREE CIRCLE

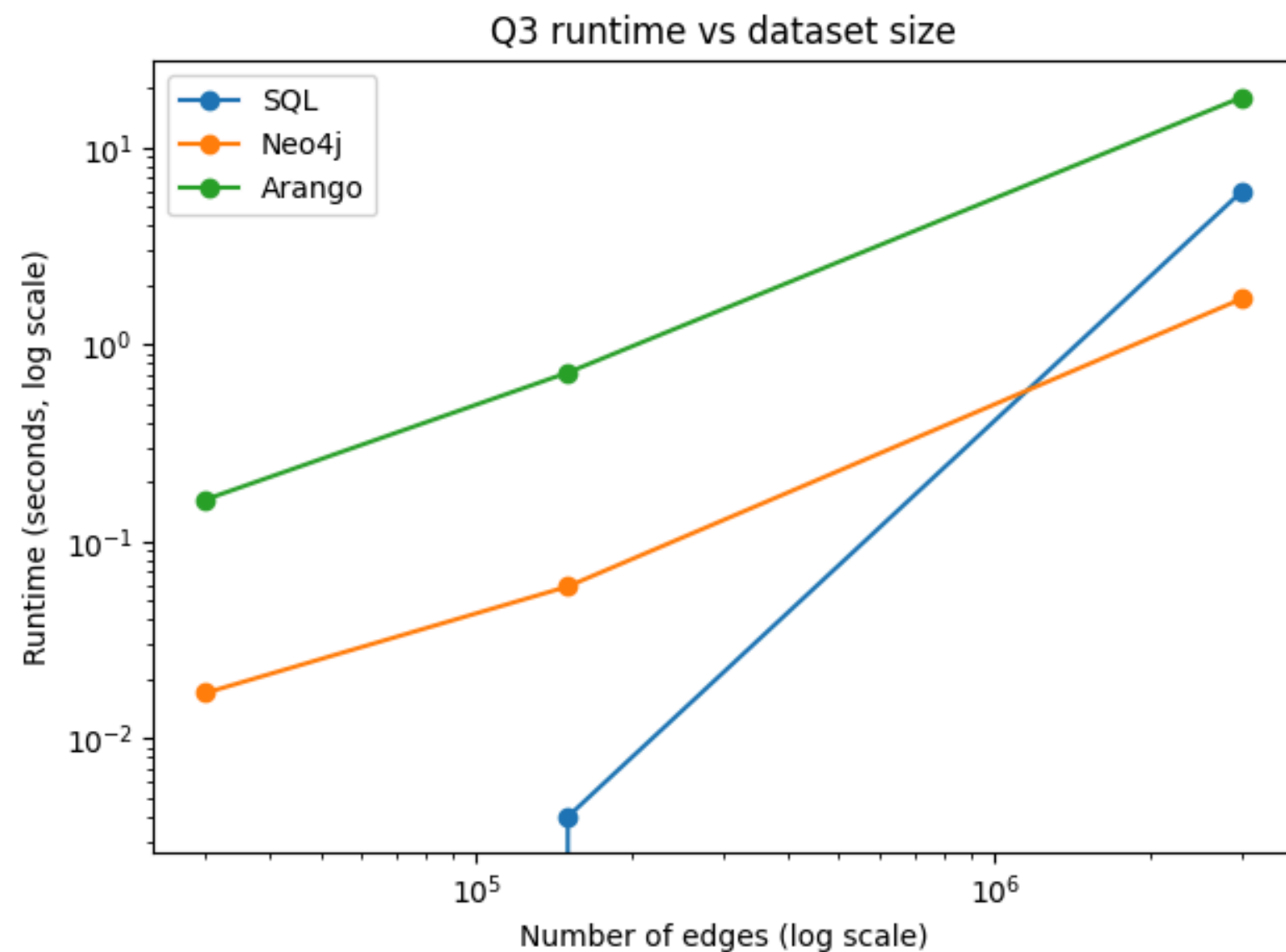


Discussion

- SQL is significantly slower, ranging from about 100 to 10,000 times slower than ArangoDB
- ArangoDB performs slightly worse than Neo4j on small datasets, but is about 100 times slower on the largest dataset

QUERY 3

COUNT HEALTHY PEOPLE AT RISK FROM CLOSE CONTACTS

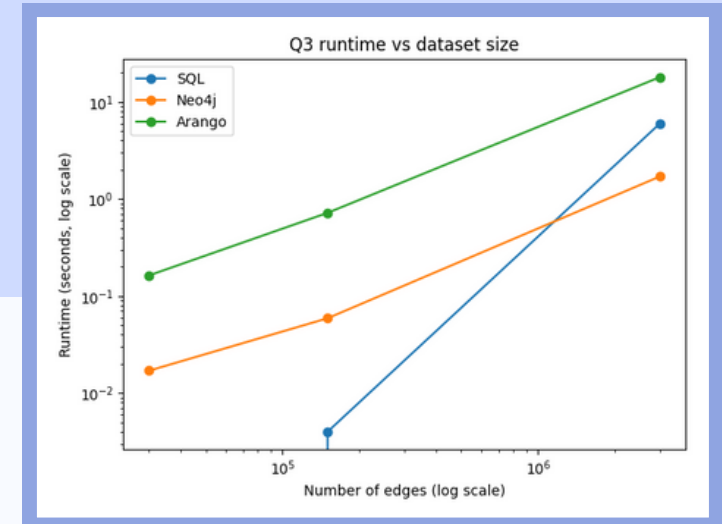


Discussion

- SQL query is faster on average than ArangoDB and Neo4j
- It consists of a simple join with some property filters, something relational databases are highly optimized for
- ArangoDB is 10 times slower

QUERY 3

COUNT HEALTHY PEOPLE AT RISK FROM CLOSE CONTACTS



SQL

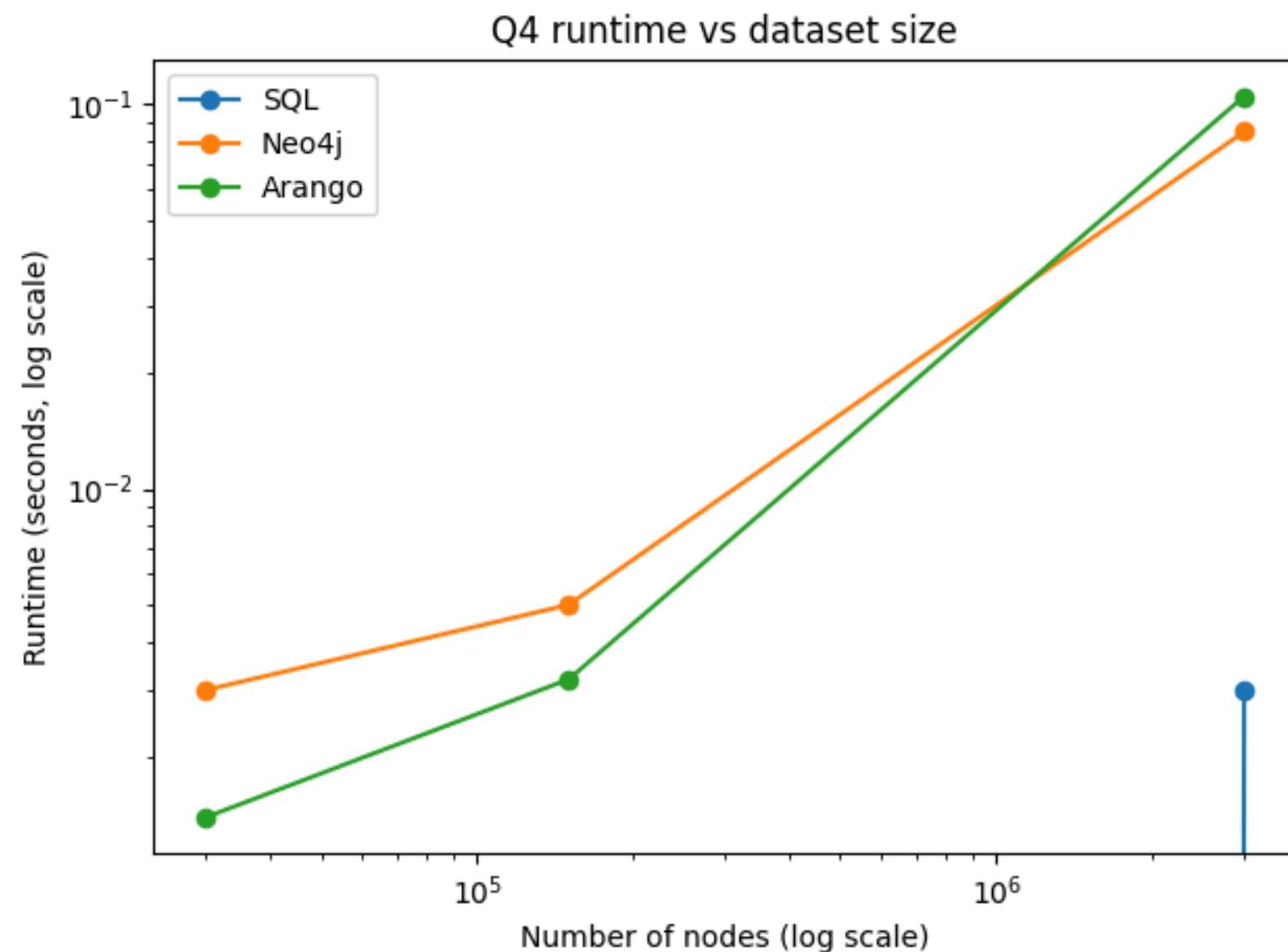
- Can use indexes to efficiently identify relevant rows and handle everything in a bulk-oriented manner
- Naturally outperforms graph databases on one step queries

Neo4j and ArangoDB

- Designed for exploring complex relationships and multi-hops traversals
- Checking direct neighbors involves pattern matching and going through edges one by one, which adds overhead

QUERY 4

COUNT COVIDED PEOPLE OF AGE 65+

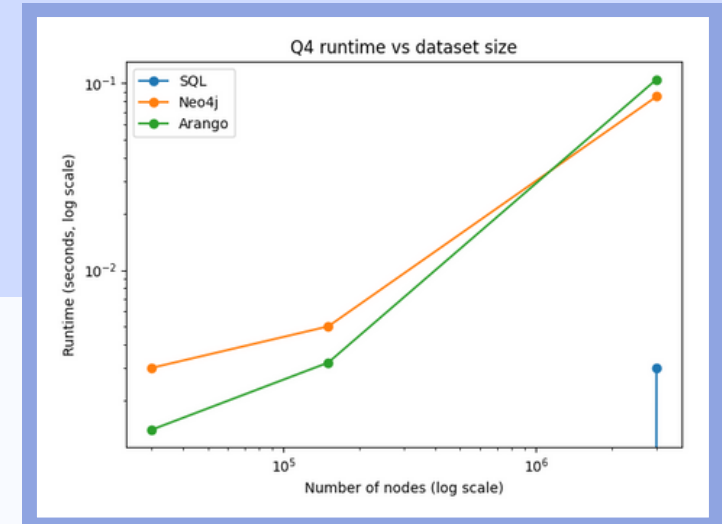


Discussion

- SQL performs particularly well on node filtering
- Neo4j is approximately 2 times slower than ArangoDB at small scale and about 2 times faster on the largest dataset

QUERY 4

COUNT COVIDED PEOPLE OF AGE 65+



SQL

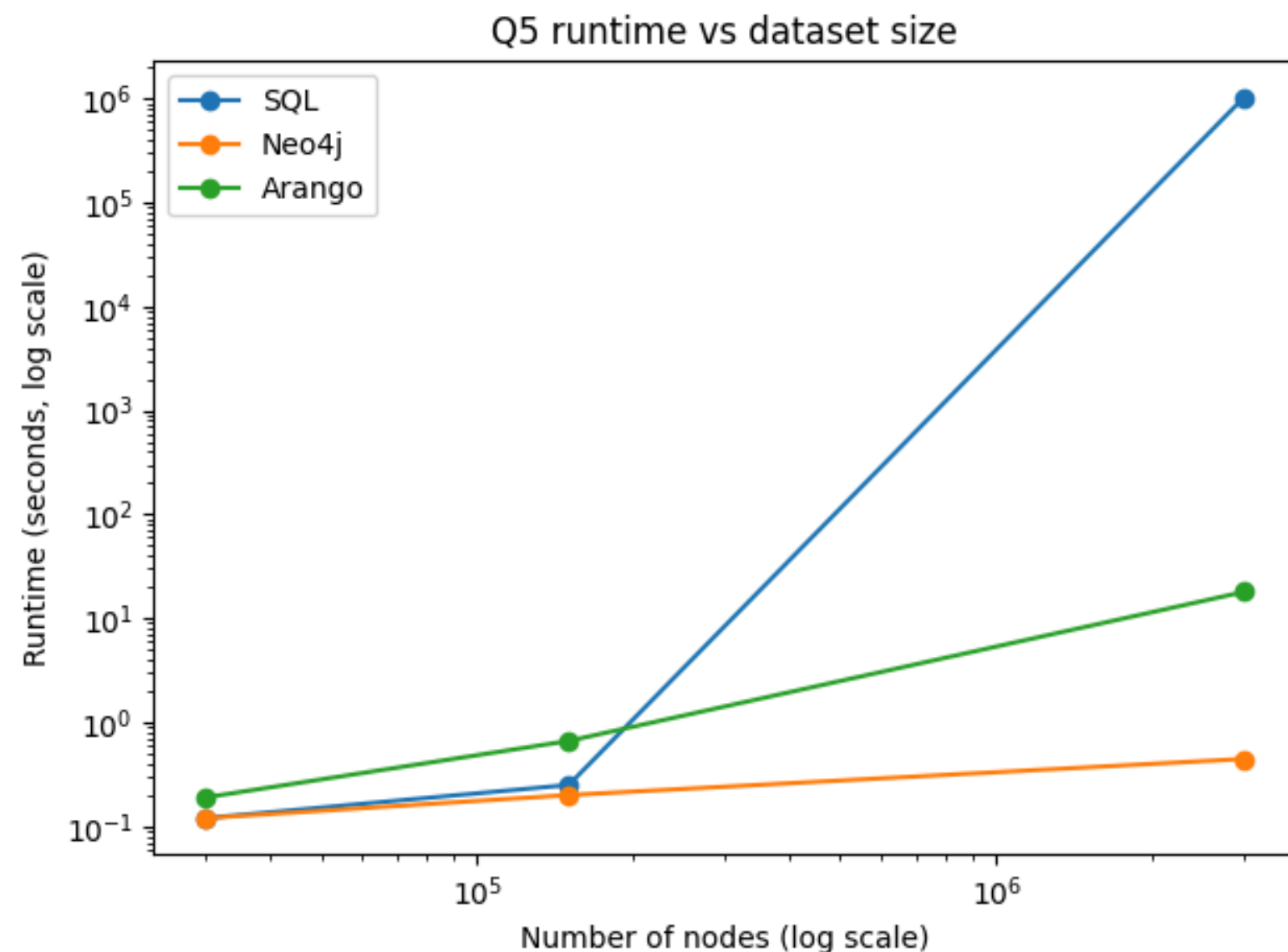
- This operation reduces to a simple select query on the nodes table
- Most efficient option when the use case does not primarily rely on relationships or graph traversals

Neo4j and ArangoDB

The additional overhead introduced by the graph-specific data structures of Neo4j could explain that small datasets run slower than ArangoDB

QUERY 5

CONTAMINATE PEOPLE IN RELATIONSHIPS WITH COVID CASES

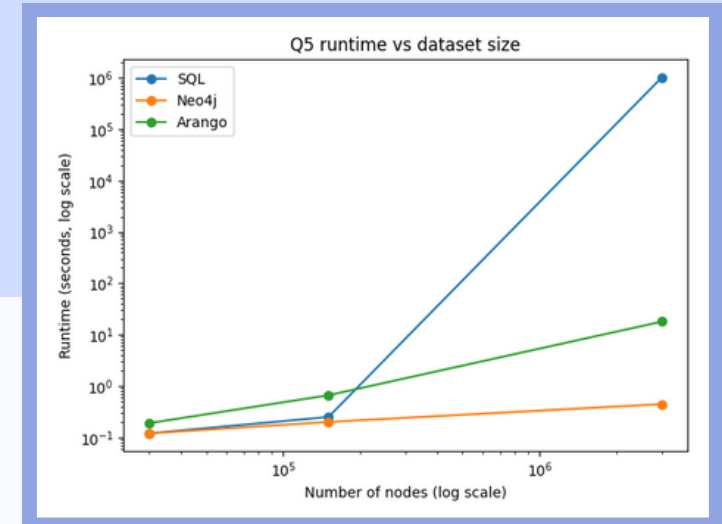


Description

- Models a propagation process over l-hop neighborhoods
- ArangoDB, Neo4j and SQL all starts with a similar runtime,
- ArangoDB ends up with a runtime 100 times slower than Neo4j
- SQL becomes 10,000,000 times slower than Neo4j

QUERY 5

CONTAMINATE PEOPLE IN RELATIONSHIPS WITH COVID CASES



SQL

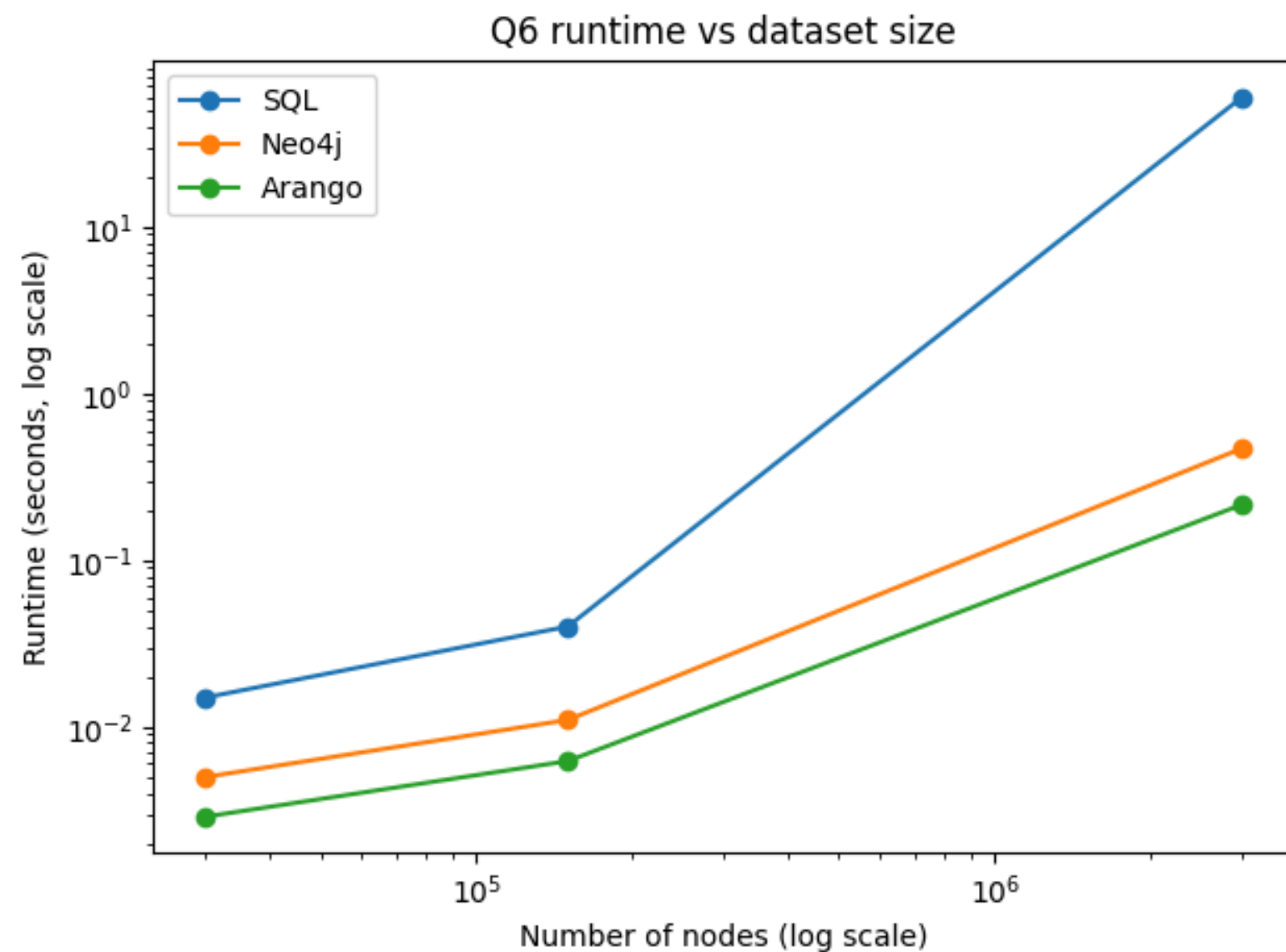
- Propagation requires costly join and update operations over large tables
- The increasing number of joins, updates, and index maintenance operations leads to a performance degradation

Neo4j and ArangoDB

- Stays linear with the size of the graph
- Specifically optimized for traversing relationships and updating connected nodes

QUERY 6

KILL PEOPLE WITH COVID

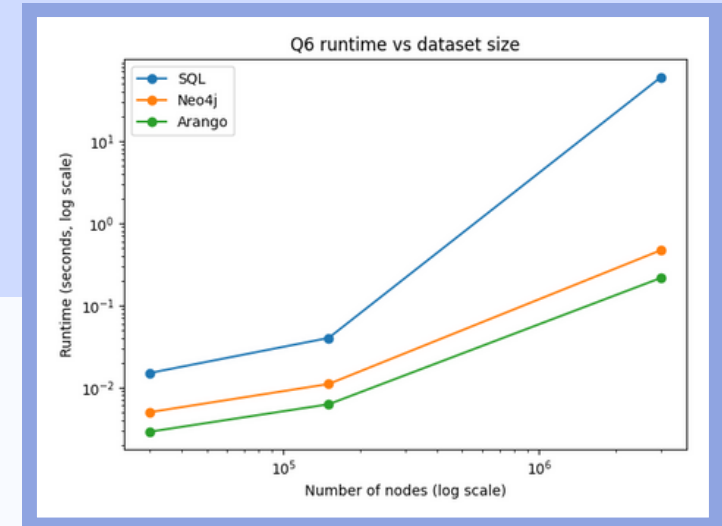


Discussion

- ArangoDB outperforms Neo4j by a factor of 2 to 5 on small datasets and about 2 times on the largest dataset
- SQL degrades on the largest dataset

QUERY 6

KILL PEOPLE WITH COVID



SQL

Perform costly join operations and bulk updates over large relational tables when deleting nodes and maintaining referential integrity, which scale poorly for graph-like deletion operations

Neo4j and ArangoDB

This comparison could be attributed to differences in internal graph representations and the way each database maintains relationships and indices during deletions.

6. CONCLUSION

CONCLUSION

SQL

Interestingly, SQL performed the best for simple operations, such as filtering nodes by properties or basic edge filtering.

For more complex queries involving lots of joins and graph-specific operations, SQL naturally started struggling.

Neo4j

Neo4j performed the best for graph operations especially, such as traversals. This was expected, since Neo4j's graph database uses native graph structures from the ground-up.

ArangoDB

ArangoDB performed better than SQL, but not as good as Neo4j. Its multi-model feature allows it to be more flexible when handling different kinds of data, and can be useful when, for example, we want to filter lots of nodes, but keep the concept of relationships.

MADE BY:

- Antoine Berthion (566199)
- Capucine Speilers (540555)
- Andrius Ežerskis (542698)
- Nha Truong (576343)

THANK YOU

FOR LISTENING

PROJECT:

Graph databases with Neo4j and
ArangoDB

2025 - 2026